📋 React Native Master Guide — Level 2 (Complete Verbatim Edition)

Virtualized Lists, Memory Recycling, Pull-to-Refresh, Sticky Section Headers, Forms & Mobile Keyboard Avoidance

EXPO V54 • NATIVEWIND / TAILWIND CSS • ES6+

📋 Level 2.1: Lists & Performance (ScrollView vs FlatList)

1. Mental Model: Why .map() in ScrollView Breaks Mobile Apps

Web lo manam normal ga `array.map()` vaadi list render chestham. Mobile lo kooda `<ScrollView>` lopalaki `items.map()` rayochu. **Kani idi 50+ items unte mobile lo disaster!**

Feature	<ScrollView>	<FlatList>
Rendering Strategy	**Eager Rendering** (All 1,000 items render into memory at once on load)	**Virtualization / Lazy Windowing** (Only visible items on screen are rendered)
RAM / Memory Usage	High (RAM spikes, app freezes or crashes on low-end phones)	**Constant & Low** (Memory is recycled as you scroll)
Best Used For	Small, fixed content (Settings page, Forms, static Profile)	**Dynamic, large datasets** (Product feeds, Chat messages, Contacts)

```
ScrollView (Heavy RAM Spike ❌):
[Item 1] [Item 2] [Item 3] ... [Item 1000] (All rendered in memory simultaneously)

FlatList (Memory Recycling ✅):
   ▲ Unmounted (Freed from RAM)
┌──────────────────────┐
│ [Item 4] (Visible)   │
│ [Item 5] (Visible)   │ <--- ONLY ~5 to 10 items kept in memory!
│ [Item 6] (Visible)   │
└──────────────────────┘
   ▼ Not rendered yet
```

2. Core FlatList Props You Must Master

1. `data` : Array of items you want to render (e.g., `products`, `users`).
2. `renderItem` : Arrow function that gets called for each item: `({ item, index }) => <YourComponent item={item} />`.
3. `keyExtractor` : Unique string ID for each item: `(item) => item.id.toString()`. *(Numbers must be converted to strings!)*
4. `ItemSeparatorComponent` : Renders a divider line between items (automatically ignores the top and bottom edges).
5. `ListEmptyComponent` : Displayed when `data = []` (no items found state).
6. `ListHeaderComponent` : Header component that scrolls naturally along with the list.

💻 Code Example: High-Performance Notification Feed

```
import React, { useState } from 'react';
import { View, Text, FlatList, Pressable } from 'react-native';
import { SafeAreaProvider, SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';

// Sample dataset
const INITIAL_NOTIFICATIONS = [
  { id: '1', title: 'Payment Received', desc: '₹2,499 credited from Kiran', time: '2m ago', type: 'success' },
  { id: '2', title: 'Security Alert', desc: 'New login detected on Chrome', time: '15m ago', type: 'warning' },
  { id: '3', title: 'Order Shipped', desc: 'Your package is out for delivery', time: '1h ago', type: 'info' },
  { id: '4', title: 'Subscription Renewed', desc: 'Pro plan renewed for 1 year', time: '3h ago', type: 'success' },
  { id: '5', title: 'Cloud Backup Complete', desc: '2.4 GB synced to drive', time: '5h ago', type: 'info' },
];

const App = () => {
  const [notifications, setNotifications] = useState(INITIAL_NOTIFICATIONS);

  // 1. Single Item Renderer Component
  const renderNotificationItem = ({ item }) => {
    const badgeBg = item.type === 'success' ? 'bg-emerald-500/20 text-emerald-400 border-emerald-500/30' :
                    item.type === 'warning' ? 'bg-amber-500/20 text-amber-400 border-amber-500/30' :
                    'bg-sky-500/20 text-sky-400 border-sky-500/30';

    return (
      <View className="bg-slate-900 border border-slate-800 p-4 rounded-2xl flex-row items-center justify-between">
        <View className="flex-1 pr-3">
          <View className="flex-row items-center gap-2 mb-1">
            <Text className="text-white font-bold text-base">{item.title}</Text>
            <Text className={`text-[10px] font-bold px-2 py-0.5 rounded-full border ${badgeBg}`}>
              {item.type.toUpperCase()}
            </Text>
          </View>
          <Text className="text-slate-400 text-xs leading-relaxed">{item.desc}</Text>
        </View>
        <Text className="text-slate-500 text-xs font-semibold">{item.time}</Text>
      </View>
    );
  };

  // 2. Custom Separator between items
  const renderSeparator = () => <View className="h-3" />;

  // 3. Header component that scrolls with list
  const renderHeader = () => (
    <View className="pb-4 pt-2">
      <View className="flex-row justify-between items-center mb-1">
        <Text className="text-2xl font-black text-sky-400">Activity Feed 📋</Text>
        <Pressable
          onPress={() => setNotifications([])}
          className="bg-slate-800 px-3 py-1.5 rounded-lg border border-slate-700 active:bg-slate-700"
        >
          <Text className="text-red-400 text-xs font-bold">Clear All</Text>
        </Pressable>
      </View>
      <Text className="text-slate-400 text-xs">
        Virtualized FlatList rendering ({notifications.length} items)
      </Text>
    </View>
  );

  // 4. Empty state when list has 0 items
  const renderEmpty = () => (
    <View className="items-center justify-center py-16">
      <Text className="text-4xl mb-3">📭</Text>
      <Text className="text-white font-bold text-lg">No Notifications</Text>
      <Text className="text-slate-400 text-xs mt-1 mb-4">You're all caught up for today!</Text>
      <Pressable
        onPress={() => setNotifications(INITIAL_NOTIFICATIONS)}
        className="bg-sky-500 px-4 py-2 rounded-xl"
      >
        <Text className="text-white font-bold text-xs">Reload Feed</Text>
```

```
      </Pressable>
    </View>
  );

  return (
    <SafeAreaProvider>
      <StatusBar style="light" backgroundColor="#090d16" />
      <SafeAreaView className="flex-1 bg-slate-950 px-4">
        <FlatList
          data={notifications}
          renderItem={renderNotificationItem}
          keyExtractor={(item) => item.id}
          ItemSeparatorComponent={renderSeparator}
          ListHeaderComponent={renderHeader}
          ListEmptyComponent={renderEmpty}
          showsVerticalScrollIndicator={false}
          contentContainerStyle={{ paddingBottom: 24 }}
        />
      </SafeAreaView>
    </SafeAreaProvider>
  );
};

export default App;
```

🗣️ Telugu + English Explanation:

• **FlatList vs ScrollView**: `ScrollView` lo 1000 items unte 1000 items ni okesari phone memory (RAM) loki load chesthundi, deeni valla app lag avthundi or crash avthundi. Kani `FlatList` lo screen meeda kanipinche 5-8 items mathrame memory lo unchi, kinda scroll chesthunte paina unna items ni unmount (recycle) chesthundi!

• **keyExtractor={(item) => item.id}**: Prathi item ki unique string key ivvali. React list items re-order aina correct ga track cheyadaniki idi use avthundi.

• **ItemSeparatorComponent**: Items madhyalo equal gap or border line pettadaniki idi vaadatharu.

• **ListEmptyComponent**: Array empty (`[]`) ayinappudu automatic ga "No data found / Empty box 📭" state ni render chesthundi.

📋 Level 2.2: Pull-to-Refresh & Infinite Scroll (Pagination)

1. Mental Model: The Instagram / Twitter Feed Experience

Real-world mobile apps do not load 10,000 items at once from an API. Instead:

- **Pull-to-Refresh**: Top nundi kindaki drag chesthe (`pull`), latest fresh data kosam fetch chesthundi.
- **Infinite Scroll (Pagination)**: User list bottom ki reach ayinappudu, automatic ga next page load chesi list kindhaki append chesthundi.

2. Key Props Breakdown

- `<RefreshControl>` : `refreshing={isRefreshing}` , `onRefresh={handleRefresh}` , `colors={['#38bdf8']}` (Android), `tintColor="#38bdf8"` (iOS).
- `onEndReached` & `onEndReachedThreshold` : `onEndReached={loadMoreData}` , `onEndReachedThreshold={0.5}` (triggers when 50% screen height away from bottom).
- `ListFooterComponent` : Renders loading spinner at list bottom.

💻 Code Example: Interactive Paginated Feed

```
import React, { useState } from 'react';
import { View, Text, FlatList, ActivityIndicator, RefreshControl } from 'react-native';
import { SafeAreaProvider, SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';

const INITIAL_DATA = [
  { id: '1', title: 'Article #1', category: 'Tech' },
  { id: '2', title: 'Article #2', category: 'Design' },
  { id: '3', title: 'Article #3', category: 'Mobile' },
  { id: '4', title: 'Article #4', category: 'Cloud' },
  { id: '5', title: 'Article #5', category: 'AI' },
  { id: '6', title: 'Article #6', category: 'DevOps' },
];

const App = () => {
  const [data, setData] = useState(INITIAL_DATA);
  const [isRefreshing, setIsRefreshing] = useState(false);
  const [isLoadingMore, setIsLoadingMore] = useState(false);

  // 1. Pull-to-Refresh Handler
  const handleRefresh = () => {
    setIsRefreshing(true);
    setTimeout(() => {
      setData(INITIAL_DATA);
      setIsRefreshing(false);
    }, 1500);
  };

  // 2. Infinite Scroll (Load More) Handler
  const handleLoadMore = () => {
    if (isLoadingMore || data.length >= 18) return;

    setIsLoadingMore(true);
    setTimeout(() => {
      const nextId = data.length + 1;
      const newItems = [
        { id: String(nextId), title: `Article #${nextId}`, category: 'Tech' },
        { id: String(nextId + 1), title: `Article #${nextId + 1}`, category: 'AI' },
        { id: String(nextId + 2), title: `Article #${nextId + 2}`, category: 'Mobile' },
      ];
      setData((prevData) => [...prevData, ...newItems]);
      setIsLoadingMore(false);
    }, 1200);
  };

  // 3. Render Single Item
  const renderItem = ({ item }) => (
    <View className="bg-slate-900 border border-slate-800 p-4 rounded-2xl flex-row justify-between items-center">
      <View>
        <Text className="text-white font-bold text-base">{item.title}</Text>
        <Text className="text-slate-400 text-xs mt-0.5">Explore modern mobile architecture</Text>
      </View>
      <View className="bg-sky-500/20 px-3 py-1 rounded-full border border-sky-500/30">
        <Text className="text-sky-400 text-xs font-bold">{item.category}</Text>
      </View>
    </View>
  );

  // 4. Footer Component (Spinner when loading more data)
  const renderFooter = () => {
    if (!isLoadingMore) return null;
    return (
      <View className="py-4 flex-row justify-center items-center gap-2">
        <ActivityIndicator size="small" color="#38bdf8" />
        <Text className="text-slate-400 text-xs font-medium">Loading more articles...</Text>
      </View>
    );
  };

  return (
    <SafeAreaProvider>
```

```
      <StatusBar style="light" backgroundColor="#090d16" />
      <SafeAreaView className="flex-1 bg-slate-950 px-4">
        <View className="py-3">
          <Text className="text-2xl font-black text-sky-400">Paginated Feed 🔄</Text>
          <Text className="text-slate-400 text-xs">Pull down to refresh • Scroll down to load more</Text>
        </View>

        <FlatList
          data={data}
          renderItem={renderItem}
          keyExtractor={(item) => item.id}
          ItemSeparatorComponent={() => <View className="h-3" />}
          ListFooterComponent={renderFooter}
          onEndReached={handleLoadMore}
          onEndReachedThreshold={0.5}
          refreshControl={
            <RefreshControl
              refreshing={isRefreshing}
              onRefresh={handleRefresh}
              colors={['#38bdf8']}
              tintColor="#38bdf8"
            />
          }
          showsVerticalScrollIndicator={false}
          contentContainerStyle={{ paddingBottom: 24 }}
        />
      </SafeAreaView>
    </SafeAreaProvider>
  );
};

export default App;
```

🗣️ Telugu + English Explanation:

• **RefreshControl**: User screen top nundi kindaki drag chesinappudu spinner show avthundi. `onRefresh` function lo `isRefreshing = true` chesi API call aipoyaka `false` chestham.

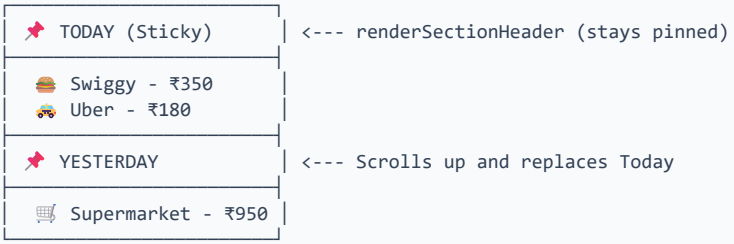
• **Platform Colors**: Android lo `colors={['#38bdf8']}` work avthundi, iOS lo `tintColor="#38bdf8"` work avthundi.

• **onEndReachedThreshold={0.5}**: `0.5` ante user list ending ki 50% screen height distance lo unnappude background lo API call trigger aypothundi.

📋 Level 2.3: <SectionList> & Sticky Headers

1. Mental Model: Why <SectionList> instead of <FlatList>?

Data **categories or groups** lo unnappudu (like Contacts A-Z, Transactions by Date [Today, Yesterday]), manaki `<SectionList>` kavali.



2. The Golden Data Structure Rule

`<SectionList>` lo `sections` prop ki pass chese array format **compulsory** ga prathi object lo `data: [...]` key tho undali:

```
const TRANSACTION_GROUPS = [
  {
    title: 'Today',
    data: [
      { id: 't1', title: 'Starbucks Coffee', amount: '-₹350' },
      { id: 't2', title: 'Metro Smart Card', amount: '-₹180' },
    ],
  },
  {
    title: 'Yesterday',
    data: [
      { id: 't3', title: 'Salary Bonus', amount: '+₹20,000' },
    ],
  },
];
```

💻 Code Example: Grouped Expense & Transaction Ledger

```
import React from 'react';
import { View, Text, SectionList } from 'react-native';
import { SafeAreaProvider, SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';

const TRANSACTION_GROUPS = [
  {
    title: 'Today',
    total: '-₹530',
    data: [
      { id: 't1', title: 'Starbucks Coffee', category: 'Food & Drinks', amount: '-₹350', isCredit: false },
      { id: 't2', title: 'Metro Smart Card', category: 'Transport', amount: '-₹180', isCredit: false },
    ],
  },
  {
    title: 'Yesterday',
    total: '+₹18,700',
    data: [
      { id: 't3', title: 'Freelance Design Payout', category: 'Income', amount: '+₹20,000', isCredit: true },
      { id: 't4', title: 'Zomato Dinner', category: 'Food & Drinks', amount: '-₹800', isCredit: false },
      { id: 't5', title: 'Fuel Pump', category: 'Vehicle', amount: '-₹500', isCredit: false },
    ],
  },
  {
    title: '25 Aug 2026',
    total: '-₹2,599',
    data: [
      { id: 't6', title: 'Annual Gym Membership', category: 'Fitness', amount: '-₹2,499', isCredit: false },
    ],
  },
];

const App = () => {
  // 1. Render Row Item
  const renderTransactionItem = ({ item }) => (
    <View className="bg-slate-900 border border-slate-800/80 p-4 rounded-2xl flex-row justify-between items-center mb-3">
      <View>
        <Text className="text-white font-bold text-base">{item.title}</Text>
        <Text className="text-slate-400 text-xs mt-0.5">{item.category}</Text>
      </View>
      <Text className={`font-extrabold text-base ${item.isCredit ? 'text-emerald-400' : 'text-slate-200'}`}>
        {item.amount}
      </Text>
    </View>
  );

  // 2. Render Sticky Section Header
  const renderSectionHeader = ({ section: { title, total } }) => (
    <View className="bg-slate-950/95 py-2.5 flex-row justify-between items-center mb-2">
      <View className="flex-row items-center gap-2">
        <View className="w-2 h-2 rounded-full bg-sky-400" />
        <Text className="text-sky-400 font-extrabold text-sm uppercase tracking-wider">
          {title}
        </Text>
      </View>
      <Text className="text-slate-400 text-xs font-semibold">
        Net: <Text className={total.startsWith('+') ? 'text-emerald-400' : 'text-slate-300'}>{total}</Text>
      </Text>
    </View>
  );

  return (
    <SafeAreaProvider>
      <StatusBar style="light" backgroundColor="#090d16" />
      <SafeAreaView className="flex-1 bg-slate-950 px-4">
        <View className="py-3 border-b border-slate-900 mb-2">
          <Text className="text-2xl font-black text-sky-400">Transactions 💳</Text>
          <Text className="text-slate-400 text-xs">Categorized & Sticky Group Headers</Text>
        </View>

        <SectionList
```

```
          sections={TRANSACTION_GROUPS}
          keyExtractor={(item) => item.id}
          renderItem={renderTransactionItem}
          renderSectionHeader={renderSectionHeader}
          stickySectionHeadersEnabled={true}
          showsVerticalScrollIndicator={false}
          contentContainerStyle={{ paddingBottom: 30 }}
        />
      </SafeAreaView>
    </SafeAreaProvider>
  );
};

export default App;
```

🗣️ Telugu + English Explanation:

• **SectionList Data Structure**: `sections` array lo prathi item lo compulsory ga `data: [...]` ane key undali.
• **stickySectionHeadersEnabled={true}**: Idi on chesthe headers screen top ki ragane akkada freeze ayyi stick avthayi. Kindaki scroll chesthunna koddi automatic ga next section header replace chesthundi.

📋 Level 2.4: Mobile Keyboard Management & Forms

1. The Mobile Problem: The 50% Screen Takeover

Mobile phones lo input field tap chesinappudu, virtual software keyboard open ayyi **screen lo 40% to 50% space ni occupy chesthundi**.

2. The 3 Essential Keyboard Tools

Tool	Purpose
`<KeyboardAvoidingView>`	Keyboard open ayinappudu automatically UI ni paiki push chesthundi. `behavior={Platform.OS === 'ios' ? 'padding' : 'height'}`
`Keyboard.dismiss()`	Blank space meeda touch chesinappudu keyboard ni close/dismiss cheyadaniki.
`keyboardShouldPersistTaps="handled"`	Keyboard open lo unnappudu "Submit" button okesari tap chesthe click trigger avthundi.

3. Crucial <TextInput> Props for Mobile

- `autoCapitalize="none"` : Emails and passwords lo first letter capital avvakunda aputhundi.
- `autoCorrect={false}` : Passwords and emails meeda red underline / autocorrect suggestions raakunda chesthundi.
- `keyboardType="email-address" | "numeric" | "phone-pad"` : Correct keyboard layout ni open chesthundi.
- `secureTextEntry={true}` : Passwords ni dots (`••••••`) ga mask chesthundi.

💻 Code Example: Robust Mobile Login & Form Validation

```
import React, { useState } from 'react';
import {
  View,
  Text,
  TextInput,
  Pressable,
  KeyboardAvoidingView,
  Platform,
  TouchableWithoutFeedback,
  Keyboard,
  ScrollView,
} from 'react-native';
import { SafeAreaProvider, SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';

const App = () => {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [showPassword, setShowPassword] = useState(false);
  const [errors, setErrors] = useState({});

  // 1. Validation Logic
  const handleLogin = () => {
    const newErrors = {};

    // Email validation
    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    if (!email.trim()) {
      newErrors.email = 'Email address is required';
    } else if (!emailRegex.test(email.trim())) {
      newErrors.email = 'Please enter a valid email';
    }

    // Password validation
    if (!password) {
      newErrors.password = 'Password is required';
    } else if (password.length < 6) {
      newErrors.password = 'Password must be at least 6 characters';
    }

    setErrors(newErrors);

    if (Object.keys(newErrors).length === 0) {
      Keyboard.dismiss();
      alert(`Success! Logging in: ${email}`);
    }
  };

  return (
    <SafeAreaProvider>
      <StatusBar style="light" backgroundColor="#090d16" />
      <SafeAreaView className="flex-1 bg-slate-950">

        {/* 1. TouchableWithoutFeedback dismisses keyboard when tapping outside */}
        <TouchableWithoutFeedback onPress={Keyboard.dismiss}>

          {/* 2. KeyboardAvoidingView lifts inputs above the software keyboard */}
          <KeyboardAvoidingView
            behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
            className="flex-1"
          >
            <ScrollView
              contentContainerStyle={{ flexGrow: 1, justifyContent: 'center' }}
              keyboardShouldPersistTaps="handled"
              className="px-6"
            >

              {/* Header */}
              <View className="mb-8">
                <Text className="text-3xl font-black text-sky-400">Welcome Back 👋</Text>
                <Text className="text-slate-400 text-sm mt-1">
```

```
          Enter your credentials to continue
        </Text>
      </View>

      {/* Form Container */}
      <View className="bg-slate-900 border border-slate-800 p-6 rounded-3xl gap-4 shadow-2xl">

        {/* Email Input */}
        <View>
          <Text className="text-slate-300 text-xs font-bold uppercase mb-2 tracking-wider">
            Email Address
          </Text>
          <TextInput
            value={email}
            onChangeText={(text) => {
              setEmail(text);
              if (errors.email) setErrors((prev) => ({ ...prev, email: null }));
            }}
            placeholder="name@example.com"
            placeholderTextColor="#64748b"
            keyboardType="email-address"
            autoCapitalize="none"
            autoCorrect={false}
            className={`bg-slate-950 text-white px-4 py-3.5 rounded-xl border ${
              errors.email ? 'border-red-500' : 'border-slate-800'
            } text-sm font-medium`}
          />
          {errors.email && (
            <Text className="text-red-400 text-xs font-medium mt-1.5 ml-1">
              ⚠️ {errors.email}
            </Text>
          )}
        </View>

        {/* Password Input */}
        <View>
          <Text className="text-slate-300 text-xs font-bold uppercase mb-2 tracking-wider">
            Password
          </Text>
          <View className="relative justify-center">
            <TextInput
              value={password}
              onChangeText={(text) => {
                setPassword(text);
                if (errors.password) setErrors((prev) => ({ ...prev, password: null }));
              }}
              placeholder="••••••••"
              placeholderTextColor="#64748b"
              secureTextEntry={!showPassword}
              autoCapitalize="none"
              autoCorrect={false}
              className={`bg-slate-950 text-white px-4 py-3.5 pr-16 rounded-xl border ${
                errors.password ? 'border-red-500' : 'border-slate-800'
              } text-sm font-medium`}
            />
            <Pressable
              hitSlop={10}
              onPress={() => setShowPassword((prev) => !prev)}
              className="absolute right-4"
            >
              <Text className="text-sky-400 font-bold text-xs">
                {showPassword ? 'HIDE' : 'SHOW'}
              </Text>
            </Pressable>
          </View>
          {errors.password && (
            <Text className="text-red-400 text-xs font-medium mt-1.5 ml-1">
              ⚠️ {errors.password}
            </Text>
          )}
        </View>

        {/* Submit Button */}
```

```
                <Pressable
                  android_ripple={{ color: '#38bdf840' }}
                  onPress={handleLogin}
                  className="bg-sky-500 active:bg-sky-600 p-4 rounded-xl items-center mt-2"
                >
                  <Text className="text-white font-bold text-base">
                    Sign In 🚀
                  </Text>
                </Pressable>

              </View>

            </ScrollView>
          </KeyboardAvoidingView>
        </TouchableWithoutFeedback>

      </SafeAreaView>
    </SafeAreaProvider>
  );
};

export default App;
```

🗣️ Telugu + English Explanation:

• **TouchableWithoutFeedback onPress={Keyboard.dismiss}**: Form lo text boxes kakunda background lo ekkada touch chesina soft keyboard automatic ga close/dismiss aypothundi.
• **KeyboardAvoidingView behavior={...}**: iOS lo `behavior="padding"` and Android lo `behavior="height"` pettali. Keyboard open avvagane inputs paiki elevate ayyi screen visibility lo untayi.
• **keyboardShouldPersistTaps="handled"**: Keyboard open lo unnappudu user direct ga "Sign In" button tap chesthe keyboard close avvadam tho paatu click event kooda immediate ga trigger avthundi.
• **autoCapitalize="none"**: Email or password lo first letter automatic ga capital aypoyi login fail avvakunda aputhundi.